

Architecture.md v1.1

Chapter 6 — Worker Agent Specification

This chapter defines the architecture, responsibilities, lifecycle, and standards for every Worker Agent in AI OS.

Worker Agents are the specialists of the system. They contain the business logic while remaining completely independent of one another.

6.1 Purpose

A Worker Agent is a self-contained module responsible for exactly one domain of expertise.

Examples include:

- Analytics
- Planner
- Business
- Trading
- Research
- Content
- Browser
- Email
- Calendar
- Vision
- Automation
- PC Control

Worker Agents never communicate with the user.

Worker Agents never perform routing.

Worker Agents never make architectural decisions.

6.2 Design Philosophy

Each Worker Agent follows five rules:

Rule 1 — One Responsibility

One agent = One domain.

Example:

Analytics Agent

Responsible for:

- Analytics

Not responsible for:

- Scheduling
 - Trading
 - Business
 - Memory
-

Rule 2 — Independence

Every agent must operate independently.

Removing one agent must not break another.

Rule 3 — Stateless Execution

Worker Agents should not retain persistent internal state.

They receive:

- Request
- Context

They return:

- Result

Persistent information belongs only in the Memory Service.

Rule 4 — Replaceability

Every Worker Agent must be replaceable.

Future versions should allow swapping an implementation without changing the Agent Manager.

Rule 5 — Testability

Every Worker Agent must be independently testable using mocked dependencies.

6.3 Standard Agent Structure

Every Worker Agent follows the same directory structure.

```
agent_name/  
  
agent.py  
config.py  
schemas.py  
models.py  
permissions.py  
tests.py  
README.md
```

Purpose:

agent.py

Main execution logic.

config.py

Agent configuration.

schemas.py

Request and response contracts.

models.py

Internal data models.

permissions.py

Permission declarations.

tests.py

Unit tests.

README.md

Documentation.

No additional mandatory files may be introduced without architecture approval.

6.4 Standard Interface

Every Worker Agent must expose the following interface.

Properties:

- Agent Name
- Description
- Supported Tasks
- Version
- Required Permissions

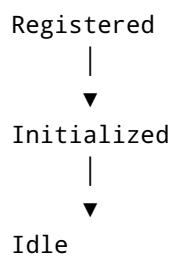
Methods:

- initialize()
- execute()
- validate_request()
- cleanup()

The Agent Manager relies on this contract.

6.5 Agent Lifecycle

Every Worker Agent follows this lifecycle.





On failure:

Executing



Failed



Return structured error



Idle

The agent remains available for future requests.

6.6 Request Processing

Every request passes through the same stages.

1. Receive request.
2. Validate input.
3. Verify permissions.
4. Retrieve required memory.
5. Execute task.
6. Return structured response.
7. Cleanup temporary data.

No stage may be skipped.

6.7 Context Rules

Agents receive only the context required.

Example:

Research Agent receives:

- Search topic
- Existing research
- Project context

It does not receive:

- Trading history
- Calendar events
- Business invoices

Minimal context improves security and performance.

6.8 Permission Model

Each Worker Agent declares required permissions.

Example:

Analytics Agent

Permissions:

- Read Analytics
- Read Memory

Denied:

- File Delete
- Trading
- PC Control

The Agent Manager enforces these permissions before execution.

6.9 Worker Agent Catalog

Planner Agent

Responsibilities:

- Task planning
 - Scheduling
 - Priorities
 - Deadlines
-

Analytics Agent

Responsibilities:

- Social media analytics
 - Reports
 - Growth metrics
 - Performance insights
-

Business Agent

Responsibilities:

- Business records
 - Clients
 - Revenue
 - Expenses
-

Trading Agent

Responsibilities:

- Market analysis
- Watchlists
- Trading journal
- Risk metrics

Does NOT execute trades automatically.

Research Agent

Responsibilities:

- Web research
 - Information gathering
 - Knowledge summaries
-

Content Agent

Responsibilities:

- Content ideas
 - Captions
 - Scripts
 - Publishing suggestions
-

Browser Agent

Responsibilities:

- Browser automation
 - Page navigation
 - Search operations
-

Email Agent

Responsibilities:

- Read emails
- Draft replies
- Categorize messages

Never sends emails without approval.

Calendar Agent

Responsibilities:

- Meetings
- Events
- Scheduling

- Availability
-

Vision Agent

Responsibilities:

- Image understanding
 - OCR
 - Document analysis
 - Object recognition
-

PC Control Agent

Responsibilities:

- Launch applications
- Manage files
- Execute safe system commands

Requires approval for destructive actions.

Automation Agent

Responsibilities:

- Workflow execution
 - Scheduled jobs
 - Task automation
-

6.10 Communication Rules

Worker Agents communicate only with:

- Agent Manager
- Memory Service (through Agent Manager-approved interfaces)

Never with:

- User
- Router
- Jarvis
- Friday

- Other Worker Agents
-

6.11 Error Handling

Every Worker Agent returns a structured response.

Possible statuses:

- SUCCESS
- FAILED
- PARTIAL_SUCCESS
- WAITING_FOR_APPROVAL
- CANCELLED

No unhandled exceptions may leave the agent.

6.12 Resource Management

Each Worker Agent must:

- Release temporary memory.
- Close open connections.
- Remove temporary files.
- Reset temporary variables.

Persistent data must already be stored through the Memory Service.

6.13 Registration Process

Before becoming available, a Worker Agent must register with the Agent Manager.

Registration includes:

- Agent Name
- Version
- Supported Tasks
- Permission Set
- Entry Point
- Health Check

Only successfully registered agents may receive requests.

6.14 Health Checks

Every Worker Agent must implement a health check.

The Agent Manager periodically verifies:

- Availability
- Dependencies
- Configuration
- Required resources

Unavailable agents are marked OFFLINE and are excluded from task assignment.

6.15 Worker Agent Principles

Every Worker Agent must satisfy:

- One responsibility.
- Independent execution.
- Stateless design.
- Standard interface.
- Structured communication.
- Permission-based execution.
- Replaceability.
- Testability.

Failure to satisfy these principles means the agent is not architecture-compliant.

End of Chapter 6

Next Chapter: Chapter 7 — Database & Persistence Architecture